Desafio 1 – LÓGICA DE PROGRAMAÇÃO – TEMA 2

MedControl – Sistema de Gerenciamento de Atendimentos de uma Clínica Médica

1. Contextualização

Um dos principais objetivos da programação é transformar problemas do mundo real em soluções computacionais organizadas. Para isso, o programador precisa compreender o problema antes de escrever o código, identificar quais informações precisam ser armazenadas, estabelecer regras de processamento e escolher estruturas adequadas para representar e manipular os dados.

Neste desafio, você deverá utilizar de maneira integrada os principais conceitos estudados durante a disciplina de *"Algoritmo e Programação".*

Diferentemente de exercícios isolados, nos quais normalmente existe uma única operação a ser realizada, este desafio apresenta um problema maior, dividido em diferentes funcionalidades que deverão funcionar de maneira integrada.

---

2. Texto-base

Uma clínica médica denominada **MedControl Saúde Integrada** realiza diariamente dezenas de atendimentos. Atualmente, parte do controle dos pacientes é realizada manualmente. Quando um paciente chega à clínica, um funcionário registra seus dados e realiza uma classificação inicial de prioridade.

Posteriormente, o paciente aguarda atendimento. Ao longo do dia, a administração também precisa consultar pacientes, verificar quantos atendimentos foram realizados, analisar prioridades, identificar pacientes que ainda aguardam atendimento e gerar algumas estatísticas.

O processo manual começou a provocar dificuldades relacionadas à organização das informações e principalmente à determinação da ordem de atendimento. Por esse motivo, a clínica decidiu desenvolver uma solução computacional experimental para organizar esse processo.

Você será responsável por desenvolver o algoritmo que realizará esse gerenciamento.

---

3. Situação-problema

O sistema deverá permitir o cadastro e gerenciamento dos pacientes que procuram atendimento durante determinado período. Cada paciente deverá possuir, no mínimo:

- código;
- nome;
- idade;
- sexo;
- prioridade;
- pressão arterial sistólica;
- pressão arterial diastólica;
- temperatura corporal;
- status do atendimento.

O sistema deverá trabalhar com quatro níveis de prioridade:

1 – Emergência
2 – Muito urgente
3 – Urgente
4 – Pouco urgente

Para fins exclusivamente didáticos deste exercício, a prioridade será informada pelo operador do sistema. O algoritmo **não deverá realizar diagnóstico médico nem determinar automaticamente a gravidade clínica do paciente**.

---

4. Constantes do sistema

Os valores utilizados para representar informações fixas deverão ser definidos utilizando constantes.

Exemplo:

```
MAX_PACIENTES = 30

EMERGENCIA = 1
MUITO_URGENTE = 2
URGENTE = 3
POUCO_URGENTE = 4

AGUARDANDO = 0
ATENDIDO = 1
```

Outras constantes poderão ser definidas pela equipe caso sejam necessárias.

Evite utilizar números sem significado diretamente nas regras do algoritmo.

---

5. Estruturas de dados obrigatórias

O sistema deverá utilizar diferentes estruturas de armazenamento.

5.1 Registro

Cada paciente deverá ser representado por meio de um **registro** contendo seus respectivos dados.

O registro deverá agrupar logicamente todas as informações relacionadas ao paciente.

5.2 Vetor de registros

Os pacientes deverão ser armazenados em um **vetor de registros**.

O vetor deverá permitir o armazenamento de todos os pacientes cadastrados durante a execução do algoritmo. A quantidade máxima deverá respeitar a constante MAX_PACIENTES.

5.3 Matriz

Uma **matriz** deverá ser utilizada para armazenar estatísticas referentes aos atendimentos.

Considere, por exemplo, que as linhas possam representar as categorias de prioridade e as colunas diferentes grupos de pacientes ou situações de atendimento. Uma possível representação seria:

Prioridade	Cadastrados	Aguardando	Atendidos
Emergência	3	1	2
Muito urgente	5	2	3
Urgente	8	3	5
Pouco urgente	10	4	6

A equipe poderá propor outra utilização coerente para a matriz, desde que sua finalidade seja claramente explicada.

6. Menu principal

O sistema deverá apresentar continuamente o seguinte menu:

MEDCONTROL – GERENCIAMENTO DE ATENDIMENTOS

1. Cadastrar paciente
2. Listar pacientes
3. Consultar paciente
4. Exibir fila de atendimento
5. Realizar atendimento
6. Exibir pacientes atendidos
7. Exibir estatísticas
8. Executar consulta recursiva
9. Encerrar sistema

A seleção das funcionalidades deverá ser implementada utilizando uma estrutura de **múltipla escolha**.
O algoritmo deverá continuar executando até que a opção de encerramento seja escolhida.

---

7. Cadastro de pacientes

O sistema deverá solicitar:

- código;
- nome;
- idade;
- sexo;
- prioridade;
- pressão arterial sistólica;
- pressão arterial diastólica;
- temperatura;
- status.

Todo novo paciente deverá inicialmente possuir o status **AGUARDANDO**. O código do paciente deverá ser único. Portanto, antes da realização de um cadastro, o algoritmo deverá verificar se o código informado já existe.
O sistema também deverá validar:

- capacidade máxima do vetor;
- idade válida;
- prioridade entre 1 e 4;
- dados numéricos inválidos;
- código duplicado.

O cadastro deverá ser implementado por meio de uma **sub-rotina**.

---

8. Consulta de pacientes

O usuário deverá poder pesquisar um paciente utilizando seu código. A operação de busca deverá ser implementada obrigatoriamente utilizando uma **função**.
Esta função deverá receber as informações necessárias por parâmetro e retornar a posição na qual o paciente foi encontrado. Caso não exista paciente com o código informado, deverá retornar um valor que permita ao programa identificar que a pesquisa não obteve resultado.

Exemplo conceitual:

Código informado: 105

Paciente encontrado.

Nome: Carlos Silva
Idade: 47
Prioridade: 2
Status: Aguardando atendimento

---

9. Fila de atendimento

O sistema deverá apresentar os pacientes que ainda aguardam atendimento. Entretanto, estes **não poderão simplesmente aparecer na ordem de cadastro**. A fila deverá ser organizada considerando os seguintes critérios:

1. maior prioridade de atendimento;
2. maior idade;
3. ordem de cadastro.

Assim, pacientes classificados como *prioridade 1* deverão aparecer antes daqueles classificados como *prioridade 2*. Caso dois pacientes possuam a mesma prioridade, o paciente de maior idade deverá aparecer primeiro. Persistindo o empate, deverá ser considerada a ordem de cadastro.

---

10. Ordenação

Para gerar corretamente a fila, será necessário implementar a **ordenação de variáveis compostas**.
O algoritmo deverá ordenar um vetor de acordo com os critérios estabelecidos. Não poderão ser utilizadas funções prontas de ordenação.
O aluno deverá implementar a lógica necessária utilizando:

- estruturas de repetição;
- estruturas condicionais;
- operadores relacionais;
- operadores lógicos;
- variáveis auxiliares.

A ordenação deverá preservar todas as informações existentes no registro do paciente.

---

11. Realização de atendimento

O sistema deverá permitir registrar que determinado paciente foi atendido. Ao selecionar essa funcionalidade, o programa deverá identificar o próximo paciente da fila.

O paciente deverá ter seu status alterado de: **AGUARDANDO** para: **ATENDIDO**

Após isso, as estatísticas deverão ser atualizadas.
Um paciente que já tenha sido atendido não poderá ser atendido novamente.

12. Operadores lógicos e relacionais

Diversas decisões do sistema exigirão a combinação de condições.

Por exemplo, para determinado processamento poderá ser necessário verificar simultaneamente:

(prioridade = **EMERGENCIA**) **E** (status = **AGUARDANDO**)
ou:
(prioridade = **EMERGENCIA**) OU (prioridade = **MUITO_URGENTE**)

O algoritmo deverá demonstrar domínio dos operadores:

- e;
- ou;
- nao;
- =;
- <>;
- >;
- <;
- >=;
- <=.

Esses operadores deverão ser utilizados em situações nas quais sejam logicamente necessários.

---

13. Estatísticas

O sistema deverá gerar estatísticas dos pacientes cadastrados.

Deverão ser apresentadas, no mínimo:

- quantidade total de pacientes;
- quantidade de pacientes aguardando;
- quantidade de pacientes atendidos;
- quantidade de pacientes por prioridade;
- quantidade de atendimentos por prioridade;
- média de idade dos pacientes;
- média de temperatura registrada;
- maior idade cadastrada;
- menor idade cadastrada;
- percentual de pacientes atendidos;
- percentual de pacientes aguardando;
- quantidade de pacientes com prioridade 1 ou 2.

Para realizar esses cálculos deverão ser utilizados:

- estruturas de repetição;
- acumuladores;
- contadores;
- operadores matemáticos;
- operadores relacionais;
- operadores lógicos.

14. Procedimentos e funções

O algoritmo deverá ser dividido em diferentes **sub-rotinas**. Entre outras possibilidades, poderão existir:

- procedimento para cadastrar paciente;
- procedimento para listar pacientes;
- função para localizar paciente;
- função para calcular quantidade de pacientes aguardando;
- procedimento para ordenar a fila;
- procedimento para realizar atendimento;
- procedimento para exibir estatísticas;
- função para calcular média;
- procedimento ou função recursiva.

A escolha entre procedimento e função deverá ser coerente com a operação executada.

---

15. Passagem de parâmetros

As sub-rotinas deverão utilizar passagem de parâmetros.

Aqui, analisar cuidadosamente quais informações deverão ser:

- recebidas pelas sub-rotinas;
- alteradas pelas sub-rotinas;
- retornadas pelas funções.

Não utilize variáveis globais apenas para evitar a passagem de parâmetros.

O objetivo é demonstrar compreensão sobre **modularização e comunicação entre diferentes partes do algoritmo**.

---

16. Variáveis locais e globais

O sistema deverá utilizar corretamente variáveis de diferentes escopos.

Variáveis que representam informações gerais do sistema poderão ser declaradas globalmente quando necessário.

Variáveis auxiliares utilizadas apenas dentro de determinada função ou procedimento deverão possuir **escopo local**.

Por exemplo, uma variável utilizada como contador temporário durante uma pesquisa não precisa necessariamente existir durante toda a execução do algoritmo.

---

17. Recursividade – Busca recursiva

O sistema deverá possuir obrigatoriamente uma funcionalidade implementada utilizando **recursividade**.

Neste desafio, deverá ser implementada uma **busca recursiva de paciente**.

A rotina receberá como parâmetros, no mínimo:

- vetor de pacientes;
- código procurado;
- posição atual;
- quantidade de pacientes cadastrados.

A lógica deverá verificar uma posição do vetor e realizar uma nova chamada recursiva caso o paciente ainda não tenha sido encontrado. A recursão deverá terminar quando:

- o paciente for encontrado;

ou

- todas as posições válidas tiverem sido verificadas.

Portanto, a solução deverá apresentar claramente:

Caso-base: condição que encerra a recursão.
Passo recursivo: chamada da própria sub-rotina utilizando a próxima posição.

Não será considerada recursividade simplesmente criar uma função que chama outra função.
Uma rotina recursiva precisa realizar uma chamada a si própria.

---

18. Recursividade

Calcular recursivamente a quantidade de pacientes que ainda aguardam atendimento.
Conceitualmente:

```
quantidadeAguardando(posição) =
  resultado da posição atual
  +
  quantidadeAguardando(próxima posição)
```

---

19. Validação dos dados

O sistema deverá possuir mecanismos de validação. Entradas inválidas não deverão simplesmente ser aceitas. O algoritmo deverá tratar situações como:

- código duplicado;
- código inexistente;
- idade negativa;
- prioridade inexistente;
- tentativa de ultrapassar a capacidade do vetor;
- tentativa de atender paciente inexistente;
- tentativa de atender paciente já atendido;
- tentativa de consultar dados quando nenhum paciente estiver cadastrado;
- opção inexistente no menu.

Sempre que possível, o sistema deverá permitir que o usuário corrija a informação sem precisar reiniciar o algoritmo.

---

20. Requisitos técnicos obrigatórios

A solução deverá utilizar obrigatoriamente:

1. variáveis simples;
2. constantes;
3. registro;
4. vetor;
5. matriz;
6. vetor de registros;
7. operadores aritméticos;
8. operadores relacionais;
9. operadores lógicos;
10. estrutura condicional simples;
11. estrutura condicional composta e/ou encadeada;
12. estrutura de múltipla escolha;
13. estrutura de repetição;
14. estruturas de repetição aninhadas;
15. contadores;

16. acumuladores;
17. algoritmo de ordenação;
18. procedimento;
19. função;
20. passagem de parâmetros;
21. variáveis locais;
22. variáveis globais;
23. recursividade;
24. validação de dados.

Os recursos deverão fazer parte da solução de maneira coerente.
Não basta utilizar determinado conceito apenas para marcar sua presença no código.

---

21. Testes obrigatórios

Testar o sistema utilizando diferentes cenários.
Entre eles:

- primeiro paciente cadastrado;
- vários pacientes cadastrados;
- pacientes com prioridades diferentes;
- pacientes com mesma prioridade;
- pacientes com mesma prioridade e idades diferentes;
- tentativa de código duplicado;
- tentativa de consultar código inexistente;
- tentativa de cadastrar prioridade inválida;
- atendimento do primeiro paciente da fila;
- tentativa de atender novamente um paciente;
- fila sem pacientes aguardando;
- execução da busca recursiva;
- consulta das estatísticas;
- tentativa de selecionar opção inexistente no menu.

Os testes deverão demonstrar que a solução funciona tanto em situações normais quanto em situações excepcionais.

---

22. Organização do algoritmo

O código deverá possuir organização compatível com a complexidade do problema.
Espera-se que o algoritmo apresente:

- indentação adequada;
- nomes significativos para variáveis;
- comentários objetivos;
- divisão em módulos;
- procedimentos e funções com responsabilidades bem definidas;
- redução de código duplicado;
- utilização adequada de constantes;
- separação entre entrada, processamento e saída.

Um código maior não deve ser tratado como um conjunto desorganizado de comandos.
Quanto maior o problema, maior deverá ser a preocupação com a **estrutura da solução**.

23. Chamada para ação

Seu desafio

A clínica **MedControl Saúde Integrada** precisa organizar seus atendimentos e abandonar o controle exclusivamente manual. Sua missão será **analisar, projetar, desenvolver e testar um algoritmo completo utilizando VisuAlg** capaz de resolver o problema apresentado. Não comece simplesmente escrevendo comandos.

Antes da implementação, analise:

- **Quais informações pertencem ao paciente?**
- **Como representar um paciente utilizando um registro?**
- **Como armazenar vários pacientes?**
- **Qual será a responsabilidade da matriz?**
- **Como localizar determinado paciente?**
- **Como determinar corretamente a ordem de atendimento?**
- **Como ordenar um vetor de registros utilizando mais de um critério?**
- **Quais funcionalidades devem ser procedimentos?**
- **Quais operações fazem sentido como funções?**
- **Quais informações deverão ser passadas como parâmetros?**
- **Quais variáveis realmente precisam possuir escopo global?**
- **Qual será o caso-base da função recursiva?**
- **Como impedir que o sistema entre em estados inválidos?**

O objetivo não é apenas chegar a uma solução que execute.

O desafio consiste em construir uma solução que demonstre **domínio da lógica utilizada**.

---

24. Apresentação e defesa da solução

Após concluir o desenvolvimento, os integrantes deverão ser capazes de explicar o funcionamento do algoritmo. Durante a apresentação, poderão ser solicitadas explicações sobre trechos específicos do código.

Exemplos:

- **"Por que você utilizou um registro aqui?"**
- **"Por que essa variável é global?"**
- **"O que aconteceria se ela fosse local?"**
- **"Por que esse parâmetro foi passado para a função?"**
- **"Como funciona a ordenação da fila?"**
- **"Como o algoritmo resolve um empate de prioridade?"**
- **"Qual é o caso-base da função recursiva?"**
- **"O que aconteceria se o caso-base fosse removido?"**
- **"Por que você utilizou E nesta condição em vez de OU?"**
- **"Onde os dados da matriz são atualizados?"**
- **"Por que esta operação foi implementada como função e não como procedimento?"**

A equipe deverá conseguir relacionar o código desenvolvido aos conceitos estudados durante a disciplina.

---

25. Critério fundamental do desafio

Este trabalho não avalia apenas se o algoritmo apresenta a saída esperada, avaliando principalmente a capacidade de **pensar algoritmicamente**.

Uma solução adequada deverá demonstrar:

Análise → Modelagem → Decomposição → Implementação → Teste → Explicação

O resultado final deverá ser um algoritmo funcional, organizado e modular, no qual seja possível identificar claramente a aplicação dos conceitos estudados durante a disciplina.

O algoritmo pertence a quem consegue explicá-lo.

Se você consegue executar o código, mas não consegue explicar sua lógica, ainda não concluiu o desafio.

Faça funcionar. Teste. Entenda. E esteja preparado para defender cada decisão tomada na construção da solução.

Entrega:

- Individual.
- Criar uma pasta **DESAFIO1** no repositório da disciplina no GitHub.
- Salvar nesta pasta o código em um único arquivo em **formato .alg**
- Salvar também um arquivo em **formato .txt** contendo o link do vídeo explicativo.
- O vídeo deve ser gravado e submetido ao YouTube, Google Drive ou outro serviço de armazenamento em Cloud e compartilhar o link.